

Clustering con K-Means

Analítica de Datos, Universidad de San Andrés

Si encuentran algún error en el documento o hay alguna duda, mandenme un mail a rodriguezf@udesa.edu.ar y lo revisamos.

1. ¿Por qué necesitamos Clustering?

Imaginá que tenés una empresa de e-commerce con miles de clientes. Todos compran en tu plataforma, pero claramente no todos son iguales: algunos compran seguido pero poco, otros compran mucho de una vez, otros compran productos caros, otros baratos, algunos solo en promociones, etc.

La pregunta es: **¿podemos agrupar automáticamente a los clientes en grupos con comportamientos similares?** Esto nos permitiría:

- Personalizar campañas de marketing para cada grupo
- Entender mejor qué tipo de clientes tenemos
- Ofrecer productos específicos a cada segmento
- Asignar recursos de manera más eficiente

El problema es que no sabemos de antemano cuántos grupos hay ni qué características definen a cada grupo. No tenemos etiquetas como en clasificación supervisada. Acá entra el **Clustering**: una técnica de aprendizaje no supervisado que agrupa observaciones similares sin necesitar etiquetas previas.

2. ¿Qué es Clustering?

Clustering es el proceso de dividir un conjunto de datos en grupos (llamados *clusters*) de manera que:

- Las observaciones **dentro de un mismo cluster** sean lo más similares posible entre sí

- Las observaciones **de diferentes clusters** sean lo más diferentes posible entre sí

Es importante entender que clustering es **aprendizaje no supervisado**: no tenemos una variable objetivo que queremos predecir. Solo tenemos datos y queremos encontrar estructura o patrones ocultos en ellos.

2.1. Aplicaciones Comunes

El clustering se usa en muchísimos contextos:

- **Marketing**: segmentación de clientes para campañas personalizadas
- **Biología**: clasificar genes o especies con características similares
- **Detección de anomalías**: identificar transacciones fraudulentas (quedan en clusters pequeños o aislados)
- **Compresión de imágenes**: agrupar colores similares para reducir el tamaño de archivos
- **Sistemas de recomendación**: agrupar usuarios con gustos similares
- **Organización de documentos**: agrupar noticias o papers por temas

3. K-Means

K-Means es probablemente el algoritmo de clustering más usado. Es simple, rápido, y funciona bien en muchos casos.

3.1. La Idea Intuitiva

La idea detrás de K-Means es la siguiente:

1. Elegís cuántos grupos querés (el famoso “K”)
2. El algoritmo coloca K puntos al azar en el espacio (estos son los *centroides* iniciales)
3. Cada observación se asigna al centroide más cercano

4. Se recalcula la posición de cada centroide como el promedio de todas las observaciones asignadas a él
5. Se repiten los pasos 3 y 4 hasta que los centroides dejen de moverse (o se muevan muy poco)

Es como si tuvieras K imanes que atraen puntos cercanos, y después se reposicionan en el centro de los puntos que atraieron, y así sucesivamente hasta que todo se estabiliza.

3.2. El Algoritmo Paso a Paso

Veamos el algoritmo de manera más formal:

1. **Inicialización:** Elegí K centroides iniciales $\mu_1, \mu_2, \dots, \mu_K$ (generalmente al azar)
2. **Asignación:** Para cada observación x_i , asignala al cluster j cuyo centroide esté más cerca:

$$c_i = \arg \min_j \|x_i - \mu_j\|^2$$

donde c_i es el cluster asignado a la observación i , y $\|\cdot\|$ es la distancia euclidiana.

3. **Actualización:** Recalculá cada centroide como el promedio de todas las observaciones asignadas a ese cluster:

$$\mu_j = \frac{1}{|C_j|} \sum_{i \in C_j} x_i$$

donde C_j es el conjunto de observaciones asignadas al cluster j .

4. **Repetición:** Repetí los pasos 2 y 3 hasta que los centroides no cambien (o cambien muy poco).

3.3. Ejemplo Manual Simplificado

Supongamos que tenemos 6 clientes con dos características: “Gasto Promedio por Compra” y “Frecuencia de Compra al Mes”:

Cliente	Gasto Promedio (\$)	Frecuencia (veces/mes)
A	20	2
B	25	3
C	100	8
D	95	9
E	22	2
F	105	10

Queremos dividirlos en $K=2$ clusters.

Paso 1 - Inicialización: Elegimos centroides iniciales al azar, digamos:

- $\mu_1 = (20, 2)$ (Cliente A)
- $\mu_2 = (100, 8)$ (Cliente C)

Paso 2 - Asignación: Calculamos la distancia de cada cliente a cada centroide y asignamos al más cercano.

Por ejemplo, para el Cliente B (25, 3):

- Distancia a μ_1 : $\sqrt{(25 - 20)^2 + (3 - 2)^2} = \sqrt{26} \approx 5,1$
- Distancia a μ_2 : $\sqrt{(25 - 100)^2 + (3 - 8)^2} = \sqrt{5650} \approx 75,2$

Cliente B está más cerca de μ_1 , entonces va al Cluster 1.

Si hacemos esto para todos:

- **Cluster 1:** A, B, E (clientes de bajo gasto y baja frecuencia)
- **Cluster 2:** C, D, F (clientes de alto gasto y alta frecuencia)

Paso 3 - Actualización: Recalculamos los centroides:

- $\mu_1 = \left(\frac{20+25+22}{3}, \frac{2+3+2}{3} \right) = (22,3, 2,3)$

- $\mu_2 = \left(\frac{100+95+105}{3}, \frac{8+9+10}{3} \right) = (100, 9)$

Paso 4 - Repetición: Volvemos al paso 2 con los nuevos centroides. En este caso, las asignaciones probablemente no cambien, entonces el algoritmo converge.

4. ¿Cómo Elegir K (el Número de Clusters)?

Esta es la pregunta del millón. A diferencia del aprendizaje supervisado, acá no tenemos un conjunto de test para validar. Entonces, ¿cómo sabemos cuántos clusters usar?

4.1. Método del Codo (Elbow Method)

La idea es probar diferentes valores de K y graficar la “inercia” (también llamada suma de distancias al cuadrado intra-cluster) para cada K. La **inercia** mide qué tan compactos son los clusters: es la suma de las distancias al cuadrado de cada punto a su centroide asignado.

$$\text{Inercia} = \sum_{i=1}^n \|x_i - \mu_{c_i}\|^2$$

Obviamente, a mayor K, menor inercia (en el extremo, si K = número de observaciones, la inercia es 0). Pero no queremos un K gigante porque pierde sentido. El truco es graficar la inercia vs K y buscar un “codo” en la curva: el punto donde agregar más clusters no reduce tanto la inercia.

Ejemplo visual: Si K=1 tenés mucha inercia, K=2 reduce bastante, K=3 reduce un poco más, pero K=4 ya no reduce tanto. El “codo” estaría en K=3.

4.2. Coeficiente de Silueta (Silhouette Score)

Otra métrica popular es el **Silhouette Score**, que mide qué tan bien está cada observación asignada a su cluster. Para cada observación i :

- a_i = distancia promedio a todas las otras observaciones en su mismo cluster (qué tan cerca está de su grupo)
- b_i = distancia promedio al cluster más cercano que no es el suyo (qué tan lejos está del grupo más próximo)

El Silhouette Score de la observación i es:

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

El score va de -1 a 1:

- Cercano a 1: la observación está bien asignada (cerca de su cluster, lejos de otros)
- Cercano a 0: está en el borde entre clusters
- Negativo: probablemente está mal asignada

El Silhouette Score promedio de todos los puntos nos da una idea de qué tan buena es la segmentación para un K dado. Elegimos el K que maximice este score.

5. Implementación en Python

Veamos cómo usar K-Means en Python con `scikit-learn`. Es súper simple.

5.1. Imports Necesarios

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.cluster import KMeans
5 from sklearn.metrics import silhouette_score
```

5.2. Aplicar K-Means

```
1 # Supongamos que X es nuestro DataFrame con las features
2 # Por ejemplo: X = df[['gasto_promedio', 'frecuencia_compra']]
3
4 # Crear el modelo con K=3
5 kmeans = KMeans(n_clusters=3, random_state=42)
6
7 # Entrenar y obtener las asignaciones de clusters
8 clusters = kmeans.fit_predict(X)
```

```

9
10 # Obtener los centroides
11 centroides = kmeans.cluster_centers_
12
13 # Agregar los clusters al DataFrame original
14 df['cluster'] = clusters

```

5.3. Encontrar el K Óptimo: Método del Codo

```

1 # Probar diferentes valores de K
2 inercias = []
3 K_range = range(1, 11)
4
5 for k in K_range:
6     kmeans = KMeans(n_clusters=k, random_state=42)
7     kmeans.fit(X)
8     inercias.append(kmeans.inertia_)
9
10 # Graficar
11 plt.figure(figsize=(10, 6))
12 plt.plot(K_range, inercias, marker='o')
13 plt.xlabel('Numero de Clusters (K)')
14 plt.ylabel('Inercia')
15 plt.title('Metodo del Codo')
16 plt.grid(True)
17 plt.show()

```

5.4. Encontrar el K Óptimo: Silhouette Score

```

1 # Calcular Silhouette Score para diferentes K
2 silhouette_scores = []
3 K_range = range(2, 11) # Minimo 2 clusters para Silhouette
4
5 for k in K_range:
6     kmeans = KMeans(n_clusters=k, random_state=42)
7     clusters = kmeans.fit_predict(X)
8     score = silhouette_score(X, clusters)
9     silhouette_scores.append(score)
10
11 # Graficar
12 plt.figure(figsize=(10, 6))
13 plt.plot(K_range, silhouette_scores, marker='o')

```

```

14 plt.xlabel('Numero de Clusters (K)')
15 plt.ylabel('Silhouette Score')
16 plt.title('Silhouette Score por K')
17 plt.grid(True)
18 plt.show()

```

6. Interpretación de Resultados

Una vez que tenés los clusters asignados, ¿qué hacés con eso?

6.1. Analizar los Centroides

Los centroides te dicen cuál es el “cliente típico” de cada cluster. Por ejemplo:

Cluster	Gasto Promedio	Frecuencia	Interpretación
0	\$25	2 veces/mes	Clientes ocasionales, bajo gasto
1	\$150	10 veces/mes	Clientes VIP, alto gasto y frecuencia
2	\$60	5 veces/mes	Clientes regulares, gasto medio

Con esta info podés:

- Diseñar campañas específicas para cada segmento
- Identificar oportunidades (ej: convertir Cluster 0 en Cluster 2)
- Asignar recursos (ej: dar atención premium a Cluster 1)

6.2. Visualizar los Clusters

Si tenés 2 o 3 dimensiones, podés graficar los clusters y ver visualmente cómo quedaron agrupados:

```

1 plt.figure(figsize=(10, 6))
2 plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap='viridis',
3             alpha=0.6, edgecolors='k')
4 plt.scatter(centroides[:, 0], centroides[:, 1],
5             marker='X', s=300, c='red', edgecolors='k',
6             label='Centroides')
7 plt.xlabel('Feature 1')

```

```
8 plt.ylabel('Feature 2')
9 plt.title('Clusters K-Means')
10 plt.legend()
11 plt.show()
```

Si tenés más de 2-3 dimensiones, podés usar técnicas de reducción de dimensionalidad como PCA (lo vemos en la próxima clase) para visualizar.

7. Ventajas y Limitaciones de K-Means

7.1. Ventajas

- **Simple y rápido:** fácil de entender e implementar
- **Escalable:** funciona bien con datasets grandes
- **Garantía de convergencia:** siempre termina (aunque no siempre en el óptimo global)
- **Interpretable:** los centroides son fáciles de analizar

7.2. Limitaciones

- **Tenés que elegir K de antemano:** no es obvio cuál es el mejor K
- **Sensible a la inicialización:** diferentes inicializaciones pueden dar diferentes resultados (por eso usamos `random_state`)
- **Asume clusters esféricos:** no funciona bien si los clusters tienen formas raras o elongadas
- **Sensible a outliers:** un punto muy alejado puede distorsionar un centroide
- **Asume que todos los clusters tienen tamaño similar:** puede fallar si hay clusters muy desbalanceados

7.3. ¿Cuándo NO Usar K-Means?

No uses K-Means si:

- Tus clusters tienen formas no esféricas (ej: clusters en forma de media luna). En ese caso, considerá algoritmos como DBSCAN.
- Tenés muchos outliers que no querés que afecten los centroides.
- Necesitás detectar el número de clusters automáticamente (K-Means requiere que vos lo especifiques).

8. Consideraciones Importantes

8.1. Normalización de Features

K-Means usa distancia euclidiana, entonces las features con rangos más grandes van a dominar el cálculo de distancia. Si tenés “Gasto” en miles de pesos (0-100,000) y “Frecuencia” en veces por mes (0-30), el gasto va a dominar completamente.

Solución: Normalizar o estandarizar las features antes de aplicar K-Means.

```
1 from sklearn.preprocessing import StandardScaler
2
3 scaler = StandardScaler()
4 X_scaled = scaler.fit_transform(X)
5
6 # Ahora usar X_scaled en lugar de X
7 kmeans = KMeans(n_clusters=3, random_state=42)
8 clusters = kmeans.fit_predict(X_scaled)
```

8.2. Múltiples Inicializaciones

Para evitar caer en mínimos locales, K-Means generalmente se corre varias veces con diferentes inicializaciones y se queda con la mejor (la que minimiza la inercia).

En `scikit-learn`, el parámetro `n_init` controla esto (por defecto es 10).

```
1 # Correr 20 inicializaciones diferentes y quedarse con la
  mejor
2 kmeans = KMeans(n_clusters=3, n_init=20, random_state=42)
```

9. Recursos Extra

- StatQuest: K-means clustering: <https://www.youtube.com/watch?v=4b5d3muPQmA>
- Documentación sklearn: <https://scikit-learn.org/stable/modules/clustering.html#k-means>